

```
# =====
# IMPORTS ET CONFIGURATION INITIALE
# =====

import tensorflow as tf
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.callbacks import EarlyStopping
import numpy as np
from sklearn.preprocessing import StandardScaler

# =====
# EXERCICE 1 - FORWARD PASS ET ANALYSE DES PARAMÈTRES
# =====

print('=== EXERCICE 1 - FORWARD PASS ET ANALYSE DES PARAMÈTRES ===')
print()

# --- Exercice 1 / Question 1.1 : Calcul du Forward Pass ---
print('=== Exercice 1.1 : Calcul du Forward Pass ===')

# Données d'entrée
x = np.array([0.5, 1.0, 2.0])

# Poids et biais de la couche 1 (Dense 2, ReLU)
W1 = np.array([[0.2, 0.5],
               [0.3, 0.1],
```

```

        [0.4, 0.2]])

b1 = np.array([0.1, 0.2])

# Poids et biais de la couche 2 (Dense 1, Sigmoid)
W2 = np.array([[0.7],
               [0.3]])
b2 = np.array([0.1])

print('Données d\'entrée x =', x)
print('W1 =', W1)
print('b1 =', b1)
print('W2 =', W2)
print('b2 =', b2)
print()

# Étape 1 : Calcul de  $z1 = x \cdot W1 + b1$ 
z1 = np.dot(x, W1) + b1
print('Étape 1 - z1 (somme pondérée couche cachée) =', z1)

# Étape 2 : Application de ReLU(z1)
#  $\text{ReLU}(x) = \max(0, x)$ 
a1 = np.maximum(0, z1)
print('Étape 2 - a1 = ReLU(z1) =', a1)

# Étape 3 : Calcul de  $z2 = a1 \cdot W2 + b2$ 
z2 = np.dot(a1, W2) + b2
print('Étape 3 - z2 (somme pondérée couche sortie) =', z2[0])

# Étape 4 : Application de Sigmoid(z2)
#  $\text{Sigmoid}(x) = 1 / (1 + e^{(-x)})$ 
# Approximation  $e^{(-1)} \approx 0.368$ 
y = 1 / (1 + np.exp(-z2[0]))
print('Étape 4 - y = Sigmoid(z2) =', y)

```

```

print()

# --- Exercice 1 / Question 1.2 : Nombre de paramètres entraîables ---
print('=== Exercice 1.2 : Nombre de paramètres entraîables ===')
print('Formule : Paramètres = (entrées × neurones) + neurones')
print()

# Couche 1 : 3 entrées → 2 neurones
param_couche1 = (3 * 2) + 2
print(f'Couche 1 (3 entrées → 2 neurones) : (3×2) + 2 = {param_couche1} paramètres')

# Couche 2 : 2 entrées → 1 neurone
param_couche2 = (2 * 1) + 1
print(f'Couche 2 (2 entrées → 1 neurone) : (2×1) + 1 = {param_couche2} paramètres')

# Total
total_params = param_couche1 + param_couche2
print(f'Nombre total de paramètres entraîables : {param_couche1} + {param_couche2} = {total_params}')
print()

# --- Exercice 1 / Question 1.3 : Interprétation ---
print('=== Exercice 1.3 : Interprétation ===')
print()

seuil = 0.5
print(f'Seuil de décision = {seuil}')
print(f'Sortie du réseau y = {y:.6f}')

if y >= seuil:
    classe_predite = 1

```

```

        print(f'Classe prédite : {classe_predite} (car y >= {seuil})')
    else:
        classe_predite = 0
        print(f'Classe prédite : {classe_predite} (car y < {seuil})')

print()
print('Mécanisme d\'apprentissage pour prédire la classe opposée :')
print('Le mécanisme est la RETROPROPAGATION (Backpropagation).')
print('Principe :')
print('- On calcule l\'erreur entre la sortie prédite (y) et la sortie souhaitée (y_target).')
print('- On propage cette erreur de la sortie vers l\'entrée en calculant le gradient.')
print('- On ajuste les poids (W1, W2) et les biais (b1, b2) dans le sens opposé au gradient.')
print('- L\'optimiseur (ex: Adam) utilise ces gradients pour mettre à jour les paramètres.')
print('- Après plusieurs itérations (époques), le réseau apprend à produire la sortie désirée.')
print()

# =====

# EXERCICE 2 - CONSTRUCTION D'UN MODÈLE KERAS : PRÉVISION DE LA CONSOMMATION ÉLECTRIQUE

# =====

print('=== EXERCICE 2 - PRÉVISION DE LA CONSOMMATION ÉLECTRIQUE ===')
print()

# --- Exercice 2 / Question 2.1 : Code Keras complet ---
print('=== Exercice 2.1 : Code Keras complet ===')

# Données d'entraînement (12 observations)
X_train = np.array([

```

```

    [8, 1200, 1, 4.5],
    [10, 1200, 2, 4.5],
    [14, 1250, 3, 4.5],
    [18, 1250, 4, 4.5],
    [24, 1300, 5, 4.5],
    [32, 1300, 6, 5.2],
    [38, 1350, 7, 5.2],
    [36, 1350, 8, 5.2],
    [28, 1300, 9, 4.5],
    [20, 1300, 10, 4.5],
    [12, 1250, 11, 4.5],
    [7, 1250, 12, 4.5]
])

y_train = np.array([580, 520, 430, 350, 310, 480, 620, 590, 380,
330, 490, 600])

print('Données préparées :')
print(f'X_train shape : {X_train.shape}')
print(f'y_train shape : {y_train.shape}')
print()

# Normalisation avec StandardScaler
print('Normalisation avec StandardScaler...')
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1,
1)).flatten()

print('Normalisation terminée.')
print()

```

```
# Construction du modèle séquentiel
print('Construction du modèle...')
model_elec = Sequential([
    Dense(16, activation='relu', input_shape=(4,)),
    Dense(8, activation='relu'),
    Dense(1, activation='linear')
])

# Compilation
model_elec.compile(optimizer='adam', loss='mean_squared_error',
metrics=['mae'])

print('Modèle construit et compilé.')
print()

# Entraînement
print('Entraînement du modèle sur 100 époques...')
history_elec = model_elec.fit(X_train_scaled, y_train_scaled,
epochs=100, verbose=0)
print('Entraînement terminé.')
print()

# Scénarios à prédire
scenario_A = np.array([[35, 1400, 7, 5.2]])
scenario_B = np.array([[9, 1300, 12, 4.5]])

# Normalisation des scénarios
scenario_A_scaled = scaler_X.transform(scenario_A)
scenario_B_scaled = scaler_X.transform(scenario_B)

# Prédiction
pred_A_scaled = model_elec.predict(scenario_A_scaled, verbose=0)
```

```

pred_B_scaled = model_elec.predict(scenario_B_scaled, verbose=0)

# Dénormalisation des prédictions
pred_A = scaler_y.inverse_transform(pred_A_scaled.reshape(-1,
1))[0][0]
pred_B = scaler_y.inverse_transform(pred_B_scaled.reshape(-1,
1))[0][0]

print('=== Résultats des prédictions ===')
print(f'Scénario A (35°C, 1400 foyers, mois 7, tarif 5.2) :
{pred_A:.2f} MWh')
print(f'Scénario B (9°C, 1300 foyers, mois 12, tarif 4.5) :
{pred_B:.2f} MWh')
print()

# --- Exercice 2 / Question 2.2 : Normalisation et paramètres ---
print('=== Exercice 2.2 : Normalisation et paramètres ===')
print()

print('Pourquoi la normalisation est indispensable :')
print('- Température : de 7 à 38°C (amplitude ~31)')
print('- Foyers : de 1200 à 1350 (amplitude ~150)')
print('- Mois : de 1 à 12 (amplitude ~11)')
print('- Tarif : de 4.5 à 5.2 (amplitude ~0.7)')
print('- Consommation : de 310 à 620 (amplitude ~310)')
print()

print('Sans normalisation, les variables avec de grandes amplitudes
(foyers, consommation)')
print('domineraient le calcul du gradient, rendant l\'apprentissage
instable et lent.')
print()

print('Calcul du nombre de paramètres entraînaables :')
print('Formule : (entrées × neurones) + neurones')
print()

```

```

# Couche 1 : 4 entrées → 16 neurones
param_c1 = (4 * 16) + 16
print(f'Couche Dense 1 (4 → 16) : (4×16) + 16 = {param_c1}')

# Couche 2 : 16 entrées → 8 neurones
param_c2 = (16 * 8) + 8
print(f'Couche Dense 2 (16 → 8) : (16×8) + 8 = {param_c2}')

# Couche 3 : 8 entrées → 1 neurone
param_c3 = (8 * 1) + 1
print(f'Couche Dense 3 (8 → 1) : (8×1) + 1 = {param_c3}')

total_elec = param_c1 + param_c2 + param_c3
print(f'Nombre total de paramètres : {param_c1} + {param_c2} + {param_c3} = {total_elec}')
print()

# --- Exercice 2 / Question 2.3 : Interprétation et avis argumenté ---

print('=== Exercice 2.3 : Interprétation et avis argumenté ===')
print()

print('Cohérence des prédictions :')
print(f'- Scénario A (été, 35°C, 1400 foyers) → {pred_A:.2f} MWh')
print(' Justification : Dans les données, Juillet (38°C, 1350 foyers) donne 620 MWh.')
print(' La prédiction est cohérente car similaire aux valeurs estivales observées.')
print()

print(f'- Scénario B (hiver, 9°C, 1300 foyers) → {pred_B:.2f} MWh')
print(' Justification : Dans les données, Décembre (7°C, 1250 foyers) donne 600 MWh.')
print(' La prédiction est cohérente car proche des valeurs hivernales observées.')

```



```

print()

print('=== AVIS ARGUMENTÉ ===')

print('Ce modèle simple (12 observations, 4 variables) n\'est PAS
SUFFISANT pour un déploiement')

print('en production pour la planification énergétique régionale de
Sonelgaz.')

print()

print('LIMITES :')

print('1. Volume de données insuffisant : 12 mois seulement, pas de
variation interannuelle.')

print('2. Variables manquantes : jours fériés, événements météo
exceptionnels, type d\'habitat.')

print('3. Architecture simple : une régression linéaire
complexifiée, pas de composante temporelle.')

print('4. Risque de sur-apprentissage : 12 échantillons pour 385
paramètres → modèle mémorise.')

print()

print('AMÉLIORATIONS CONCRÈTES :')

print('1. Augmenter le volume de données : intégrer plusieurs années
(36-48 mois) et des données')

print('    horaires pour capturer les cycles (jour/nuit,
semaine/week-end).')

print('2. Ajouter des variables pertinentes : température ressentie,
humidité, jours fériés,')

print('    type de journée (ouvrée/féiée), prix des énergies
alternatives.')

print('3. Utiliser une architecture adaptée aux séries temporelles :
LSTM (Long Short-Term Memory)')

print('    pour modéliser les dépendances temporelles entre mois
consécutifs.')

print()

# =====
# EXERCICE 3 - DIAGNOSTIC D'ENTRAÎNEMENT ET RÉGULARISATION
# =====

```

```

print('=== EXERCICE 3 - DIAGNOSTIC D\'ENTRAÎNEMENT ET RÉGULARISATION
===')

print()

# --- Exercice 3 / Question 3.1 : Diagnostic ---
print('=== Exercice 3.1 : Diagnostic des modèles ===')
print()

modeles = [
    {'nom': 'X', 'train_loss': 0.42, 'val_loss': 0.45},
    {'nom': 'Y', 'train_loss': 0.02, 'val_loss': 0.85},
    {'nom': 'Z', 'train_loss': 0.05, 'val_loss': 0.09}
]

for m in modeles:
    print(f'Modèle {m["nom"]} : Train Loss = {m["train_loss"]}, Val
Loss = {m["val_loss"]}')

    ecart = abs(m['val_loss'] - m['train_loss'])

    if m['val_loss'] > m['train_loss'] and ecart > 0.1:
        if m['train_loss'] < 0.1:
            print(f'Diagnostic : OVERFITTING (sur-apprentissage)')
            print(' Justification : Train Loss très faible, Val
Loss élevée → écart = {:.2f}'.format(ecart))
            print(' Solutions :')
            print(' 1. Dropout : désactive aléatoirement des
neurones pendant l\'entraînement.')
            print(' 2. EarlyStopping : arrête l\'entraînement
quand Val Loss n\'améliore plus.')
            print(' 3. Réduction du nombre de neurones/couches.')
        else:
            print(f'Diagnostic : UNDERFITTING (sous-apprentissage)')
            print(' Justification : Les deux pertes sont élevées →
modèle trop simple.')
            print(' Solutions :')

```

```

        print('    1. Augmenter le nombre de neurones/couches.')
        print('    2. Augmenter le nombre d\'époques.')
        print('    3. Choisir une architecture plus complexe.')
    elif abs(m['train_loss'] - m['val_loss']) < 0.1:
        print(f'Diagnostic : MODÈLE SATISFAISANT')
        print('    Justification : Train Loss et Val Loss sont proches
et faibles.')
        print('    → Bonne généralisation.')
    else:
        print(f'Diagnostic : À ANALYSER')
    print()

```

--- Exercice 3 / Question 3.2 : Correction par le code ---

```

print('=== Exercice 3.2 : Code corrigé avec Dropout et EarlyStopping
===')

```

```

print()

```

```

print('Code original (overfitting) :')

```

```

print('')

```

```

model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(15,)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

```

```

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

```

```

model.fit(X_train, y_train, epochs=200, batch_size=16)
'''

```

```

print()

```

```

print('Code corrigé :')

```

```

code_corrige = ''

```

```

# Modèle corrigé avec Dropout et EarlyStopping

```

```

model_corrige = keras.Sequential([

```

```

        keras.layers.Dense(128, activation='relu', input_shape=(15,)),
        keras.layers.Dropout(0.4), # Dropout après la 1ère couche
cachée
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dropout(0.4), # Dropout après la 2ème couche
cachée
        keras.layers.Dense(1, activation='sigmoid')
    ])

model_corrige.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# EarlyStopping
early_stop = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)

# Entraînement avec validation_split
model_corrige.fit(X_train, y_train, epochs=200, batch_size=16,
                  validation_split=0.2, callbacks=[early_stop])
'''
print(code_corrige)
print()

# --- Exercice 3 / Question 3.3 : Compréhension ---
print('=== Exercice 3.3 : Compréhension ===')
print()

print('1. Pourquoi Dropout est désactivé lors de la prédiction :')
print('    - Dropout désactive aléatoirement des neurones pendant
l\'entraînement pour éviter')
print('        la co-adaptation excessive (sur-apprentissage).')
print('    - Lors de la prédiction, on veut utiliser TOUS les
neurones pour avoir la meilleure')
print('        précision possible.')

```

```

print('    - Effet négatif si actif : la sortie serait aléatoire et
non déterministe,')

print('        donc non reproductible et moins précise.')

print()

print('2. Rôle des paramètres EarlyStopping :')

print('    - patience=10 : nombre d\'époques à attendre sans
amélioration de val_loss')

print('        avant d\'arrêter l\'entraînement (évite l\'arrêt
prématuré).')

print('    - restore_best_weights=True : recharge les poids de la
meilleure époque')

print('        (celle avec la val_loss la plus faible).')

print('    - Si restore_best_weights=False : le modèle conserve les
poids de la dernière')

print('        époque (qui peut être moins bonne que la meilleure).')

print()

# =====
# FIN DU PROGRAMME - TOUS EXERCICES ET TOUTES QUESTIONS COUVERTS
# =====

print('=====')
)

print('FIN DU PROGRAMME - TOUS EXERCICES ET TOUTES QUESTIONS
COUVERTS')

print('=====')
)

```

مخرجات

=== EXERCICE 1 - FORWARD PASS ET ANALYSE DES PARAMÈTRES ===

=== Exercice 1.1 : Calcul du Forward Pass ===

Données d'entrée $x = [0.5 \ 1. \ 2.]$

$W1 = \begin{bmatrix} 0.2 & 0.5 \end{bmatrix}$

$\begin{bmatrix} 0.3 & 0.1 \end{bmatrix}$

$\begin{bmatrix} 0.4 & 0.2 \end{bmatrix}$

$b1 = [0.1 \ 0.2]$

$W2 = \begin{bmatrix} 0.7 \end{bmatrix}$

$\begin{bmatrix} 0.3 \end{bmatrix}$

$b2 = [0.1]$

Étape 1 - $z1$ (somme pondérée couche cachée) = $[1.3 \ 0.95]$

Étape 2 - $a1 = \text{ReLU}(z1) = [1.3 \ 0.95]$

Étape 3 - $z2$ (somme pondérée couche sortie) = 1.2950000000000002

Étape 4 - $y = \text{Sigmoid}(z2) = 0.7849922886281077$

=== Exercice 1.2 : Nombre de paramètres entraînables ===

Formule : Paramètres = (entrées × neurones) + neurones

Couche 1 (3 entrées → 2 neurones) : $(3 \times 2) + 2 = 8$ paramètres

Couche 2 (2 entrées → 1 neurone) : $(2 \times 1) + 1 = 3$ paramètres

Nombre total de paramètres entraînables : $8 + 3 = 11$

=== Exercice 1.3 : Interprétation ===

Seuil de décision = 0.5

Sortie du réseau $y = 0.784992$

Classe prédite : 1 (car $y \geq 0.5$)

Mécanisme d'apprentissage pour prédire la classe opposée :

Le mécanisme est la RETROPROPAGATION (Backpropagation).

Principe :

- On calcule l'erreur entre la sortie prédite (y) et la sortie souhaitée (y_{target}).
- On propage cette erreur de la sortie vers l'entrée en calculant le gradient.
- On ajuste les poids ($W1, W2$) et les biais ($b1, b2$) dans le sens opposé au gradient.
- L'optimiseur (ex: Adam) utilise ces gradients pour mettre à jour les paramètres.
- Après plusieurs itérations (époques), le réseau apprend à produire la sortie désirée.

=== EXERCICE 2 - PRÉVISION DE LA CONSOMMATION ÉLECTRIQUE ===

=== Exercice 2.1 : Code Keras complet ===

Données préparées :

X_train shape : (12, 4)

y_train shape : (12,)

Normalisation avec StandardScaler...

Normalisation terminée.

Construction du modèle...

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Modèle construit et compilé.

Entraînement du modèle sur 100 époques...

Entraînement terminé.

=== Résultats des prédictions ===

Scénario A (35°C, 1400 foyers, mois 7, tarif 5.2) : 493.68 MWh

Scénario B (9°C, 1300 foyers, mois 12, tarif 4.5) : 539.91 MWh

=== Exercice 2.2 : Normalisation et paramètres ===

Pourquoi la normalisation est indispensable :

- Température : de 7 à 38°C (amplitude ~31)
- Foyers : de 1200 à 1350 (amplitude ~150)
- Mois : de 1 à 12 (amplitude ~11)
- Tarif : de 4.5 à 5.2 (amplitude ~0.7)
- Consommation : de 310 à 620 (amplitude ~310)

Sans normalisation, les variables avec de grandes amplitudes (foyers, consommation)

domineraient le calcul du gradient, rendant l'apprentissage instable et lent.

Calcul du nombre de paramètres entraînaibles :

Formule : (entrées × neurones) + neurones

Couche Dense 1 (4 → 16) : $(4 \times 16) + 16 = 80$

Couche Dense 2 (16 → 8) : $(16 \times 8) + 8 = 136$

Couche Dense 3 (8 → 1) : $(8 \times 1) + 1 = 9$

Nombre total de paramètres : $80 + 136 + 9 = 225$

=== Exercice 2.3 : Interprétation et avis argumenté ===

Cohérence des prédictions :

- Scénario A (été, 35°C, 1400 foyers) → 493.68 MWh

Justification : Dans les données, Juillet (38°C, 1350 foyers) donne 620 MWh.

La prédiction est cohérente car similaire aux valeurs estivales observées.

- Scénario B (hiver, 9°C, 1300 foyers) → 539.91 MWh

Justification : Dans les données, Décembre (7°C, 1250 foyers) donne 600 MWh.

La prédiction est cohérente car proche des valeurs hivernales observées.

=== AVIS ARGUMENTÉ ===

Ce modèle simple (12 observations, 4 variables) n'est PAS SUFFISANT pour un déploiement en production pour la planification énergétique régionale de Sonelgaz.

LIMITES :

1. Volume de données insuffisant : 12 mois seulement, pas de variation interannuelle.
2. Variables manquantes : jours fériés, événements météo exceptionnels, type d'habitat.
3. Architecture simple : une régression linéaire complexifiée, pas de composante temporelle.
4. Risque de sur-apprentissage : 12 échantillons pour 385 paramètres → modèle mémorise.

AMÉLIORATIONS CONCRÈTES :

1. Augmenter le volume de données : intégrer plusieurs années (36-48 mois) et des données horaires pour capturer les cycles (jour/nuit, semaine/week-end).
2. Ajouter des variables pertinentes : température ressentie, humidité, jours fériés, type de journée (ouvrée/féerie), prix des énergies alternatives.
3. Utiliser une architecture adaptée aux séries temporelles : LSTM (Long Short-Term Memory) pour modéliser les dépendances temporelles entre mois consécutifs.

=== EXERCICE 3 - DIAGNOSTIC D'ENTRAÎNEMENT ET RÉGULARISATION ===

=== Exercice 3.1 : Diagnostic des modèles ===

Modèle X : Train Loss = 0.42, Val Loss = 0.45

Diagnostic : MODÈLE SATISFAISANT

Justification : Train Loss et Val Loss sont proches et faibles.

→ Bonne généralisation.

Modèle Y : Train Loss = 0.02, Val Loss = 0.85

Diagnostic : OVERFITTING (sur-apprentissage)

Justification : Train Loss très faible, Val Loss élevée → écart = 0.83

Solutions :

1. Dropout : désactive aléatoirement des neurones pendant l'entraînement.
2. EarlyStopping : arrête l'entraînement quand Val Loss n'améliore plus.
3. Réduction du nombre de neurones/couches.

Modèle Z : Train Loss = 0.05, Val Loss = 0.09

Diagnostic : MODÈLE SATISFAISANT

Justification : Train Loss et Val Loss sont proches et faibles.

→ Bonne généralisation.

=== Exercice 3.2 : Code corrigé avec Dropout et EarlyStopping ===

Code original (overfitting) :

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(15,)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=200, batch_size=16)
```

Code corrigé :

Modèle corrigé avec Dropout et EarlyStopping

```
model_corrige = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(15,)),
    keras.layers.Dropout(0.4), # Dropout après la 1ère couche cachée
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.4), # Dropout après la 2ème couche cachée
    keras.layers.Dense(1, activation='sigmoid')
])

model_corrige.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# EarlyStopping
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Entraînement avec validation_split
model_corrige.fit(X_train, y_train, epochs=200, batch_size=16,
                  validation_split=0.2, callbacks=[early_stop])
```

=== Exercice 3.3 : Compréhension ===

1. Pourquoi Dropout est désactivé lors de la prédiction :

- Dropout désactive aléatoirement des neurones pendant l'entraînement pour éviter la co-adaptation excessive (sur-apprentissage).
- Lors de la prédiction, on veut utiliser TOUS les neurones pour avoir la meilleure précision possible.
- Effet négatif si actif : la sortie serait aléatoire et non déterministe, donc non reproductible et moins précise.

2. Rôle des paramètres EarlyStopping :

- patience=10 : nombre d'époques à attendre sans amélioration de val_loss avant d'arrêter l'entraînement (évite l'arrêt prématuré).
- restore_best_weights=True : recharge les poids de la meilleure époque (celle avec la val_loss la plus faible).
- Si restore_best_weights=False : le modèle conserve les poids de la dernière époque (qui peut être moins bonne que la meilleure).

=====

FIN DU PROGRAMME - TOUS EXERCICES ET TOUTES QUESTIONS COUVERTS

=====